

seen test cases by instLab 2: CS527

Release date: 7th August 2026

Submission date: 14th August 2026, 9AM.

Problem Statement:

Simulator a mini-computer system.

Users can write a program in a simple programming language, as specified in the language specification section. A compiler will compile the program, and will generate a byte code. Byte code will be read by a computer system, and will initialise a memory. The processor will read the instruction one by one to execute the program, and will modify the memory. At the end, memory will be written to the computer system in a file.

Finally, compilation and computer system should be invoked by a single main file, and executed as a single executable.

All the changes with respect to Lab 1 is done in [blue](#)

[Assume all the registers in the architecture are of 32 bits. Memory is byte addressable,](#)

Language specification:

Arithmetic operations

Language supports following basic math operations : + - / *

Dest Variable = <Operand 1 variable> <operation> <Operand 2 variable>

[Dest Variable = <Operand 1 variable> <operation> <constant value>](#)

[Example:](#)

[x1 = x2 + 10](#)

[x1 = x2 - 20](#)

[The maximum value of constant could be 255.](#)

[Every addition/subtraction operation updates the following four global flags: Z N C V](#)

[Z is 1, if result of operation is 0](#)

[N is 1, if MSB of result of operation is 1](#)

[C is 1, {For addition: if the unsigned result is strictly less than either of the unsigned inputs}. For subtraction: if operand 1 is bigger than operand 2.](#)

V is 1, {for addition: if both operands sign is same, and sign of result is different than operand's sign.} {For subtraction: if the operands have different signs, and the result's sign matches the second operand's original sign}

Memory operations: Read and Write

Read <variable> <address>

Write <variable> <address> //address is a constant

Support of square brackets.

Dest Variable = [Operand 2 variable] //memory read.

[Dest Variable address] = Operand 2 variable //memory write to given address stored in a variable.

Operand could be a variable or a constant.

Example:

x1 = [x2]

[x2] = x1;

x1 = [0]

The maximum value of constant can be 255.

Read and Write should be considered legacy instruction. Should not be used in lab 2 onwards - though compiler should still be able to compile them.

Control operations: ~~No control operation supported in today's lab.~~

Language support labels. Rules:

- a) Labels have to start with . character
- b) They should be the first character of any line (no space or tab should prefix them).
- c) Label will alphanumeric string of characters. No spacial symbol, no :
- d) Line having label will have no other expression

Example:

.label1

.loop

Control operations:

B{Suffix} <label>

Suffix given in table below:

Example:

BEQ .loop

BGE .label1

Code	Suffix	Flags	Meaning
0000	EQ	Z set	equal
0001	NE	Z clear	not equal
0010	CS	C set	unsigned higher or same
0011	CC	C clear	unsigned lower
0100	MI	N set	negative
0101	PL	N clear	positive or zero
0110	VS	V set	overflow
0111	VC	V clear	no overflow
1000	HI	C set and Z clear	unsigned higher
1001	LS	C clear or Z set	unsigned lower or same
1010	GE	N equals V	greater or equal
1011	LT	N not equal to V	less than
1100	GT	Z clear AND (N equals V)	greater than
1101	LE	Z set OR (N not equal to V)	less than or equal
1110	AL	(ignored)	always

Table is taken from [\[REF1\]](#)

More information about these condition flags [\[ARM\]](#)

Data movement operation:

<Dest Variable> = <constant value>

Variable names: x0 to x255.

Constant value: 0 to 255

Support for comments. Any character following % will be ignored by parser.

Example: Sum of an array. Assume size of Array is stored at address 0, elements stored at 0x4 onwards location. Final result will be stored at address = size * 4 + 4.

```

x1 = 0    % index
x2 = 0    % sum
x3 = 0    % array address
x4 = [x3] % read the size of array
.loopback

```

```

x15 = x1 - x4  % compare
BEQ .exit
x3 = x3 + 4    % increase the address
x6 = [x3]      % load the array data
x2 = x2 + x6   % update sum
x1 = x1 + 1    % increment index
BAL .loopback
.exit
[x3] = x2

```

Example bytecode output of the above program is (written in hex)

```

F 1 0 0
F 2 0 0
F 3 0 0
5 4 0 3
2 F 1 4
10 0 0 6
9 3 3 4
5 6 0 3
1 2 2 6
9 1 1 1
1E 0 0 FA
6 3 0 2

```

Byte code specification

Compiler generates a byte code for every operation and finally the byte code is stored in file called "program.byte".

Format of each line of byte code

<opcode> <dest register> <operand 1> <operand 2>

For branch instructions:

<opcode> <0> <0> <relative offset of jump location>

Relative offset could be positive or negative, such that:

next instruction address = current instruction address + offset.

Operation	Opcode if second operand is a variable	Opcode if second operand is a constant
Add	0x01	0x09
Subtract	0x02	0x0A

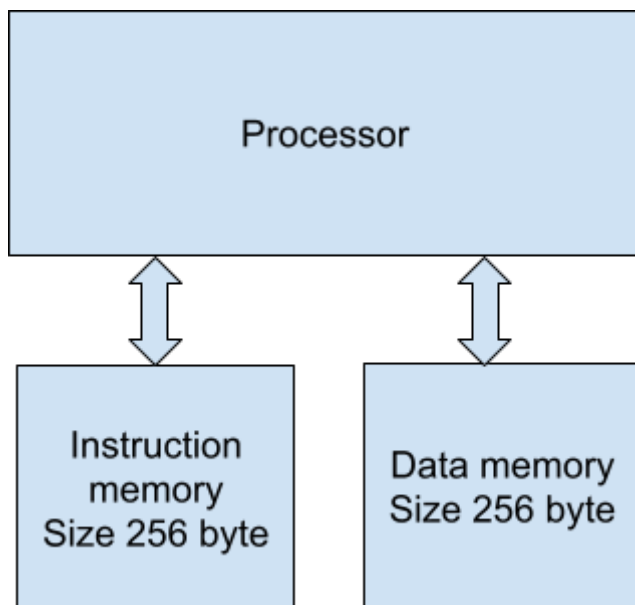
Multiply	0x03	0x0B
Divide	0x04	0x0C
Memory read	0x05	0x0C
Memory write	0x06	0x0E
Data movement	0x07	0x0F
Branch	0x10 + code given in branch table	

In case of memory read, write and data movement operations - operand 1 is 0.

Note that

- Every line has four bytes
- The maximum value for each byte is 255.

Processor specification



Processor has four functions:

reset()
fetch()
decode()

execute()

Following data elements are there in processor:

Int Register[256]

Int PC, opcode, dest, src1, src2

extern char Instruction[256], Data[4096];

Int end_of_simulation = 0

It has a global array of registers of size 256. And a global variable named PC. [Also global flags N Z C V](#)

Reset() initializes each element of register array to 0 as well initialize PC to 0

fetch(): reads four bytes from the PC address of instruction memory and stores in opcode, dest, src1, and src2 respectively

decode() : void and empty function at this time

execute(): based on opcode, it performs the operations. Once see 0 opcode, set

end_of_simulation = 1

Memory

Memory is coded as a separate 'c' file.

Char Instruction[256];

Char Data[4096];

Assumes program.byte and data.byte present in the directory. The memory is byte addressable which means when you read instructions from char array will read the instruction byte-by-byte.

[data.byte and program.byte are stored as an array of hex bytes. In each line four bytes in hex format \(without prefix 0x\) separated by space are stored](#)

[For example -1 is represented as](#)

[FF FF FF FF](#)

initialize() {

//read the data from "program.byte" and populate Instruction memory

//read the data from "data.byte" and populate data memory.

}

finalize() {

//Write data.byte

}

Main file

```
main(int argc, char** argv) {  
  
    compile();  
    initialise();  
    reset();  
  
    while(end_of_simulation) {  
        fetch();  
        decode();  
        execute();  
    }  
  
    finalize();  
  
}
```

It is suggested that the program file is passed as argument to your final executable - using `argc` and `argv`.

Directory structure

Following code files should be there:

```
main.c  
compiler.c  
compiler.h  
processor.c  
processor.h  
memory.c  
memory.h  
Makefile
```

Some tutorials on multi-file compilation:

<https://www.codewithc.in/tutorials/multi-file-programs>

https://www.w3schools.com/c/c_organize_code.php

You can search more on the internet

Makefile tutorial:

<https://makefiletutorial.com/>

FAQs

Example user level programs:

1. Sum of N numbers where N is fixed at program time
2. Multiply two complex numbers
3. Find the determinant of a 3x3 matrix.

Following can be done when branch instructions are introduced.

4. Sum of N numbers where N is given at run time.
 - a. Four bytes starting from 0x0 stores N
 - b. Next 4N bytes store the N numbers.
5. FIR filter response:
 - a. Input start at 0x0 of data memory. First four bytes tells size of array, say N. Next 4N bytes are input array.
 - b. Output is stored at 0x10.
 - c. Weights are part of the program - as constants.

Data size

- In this assignment, data size is not specified. ~~You can assume each operation is 8 bits, or 32 bits. 32-bit operation size is preferable.~~ Data size is always 32 bit.

Submission Instructions:

- Submit as a single zipped file - containing all your code, and test programs.